

Simulation Semantics

Simulators

- Race conditions
 - Verilog/SystemVerilog natively parallel
 - Event based
 - Sequential execution model unrolling the parallelism
 - Sequential order of handling the events
- Delta cycles
 - <https://vhdlwhiz.com/delta-cycles-explained/>

```
module racec;  
    int writeme;  
  
    initial begin  
        writeme = 1;  
    end  
  
    initial begin  
        writeme = 2;  
    end  
endmodule
```

Simulation

- Determine state for given discrete timestep
- Simulating every time step is expensive
- No signal changes most of the time

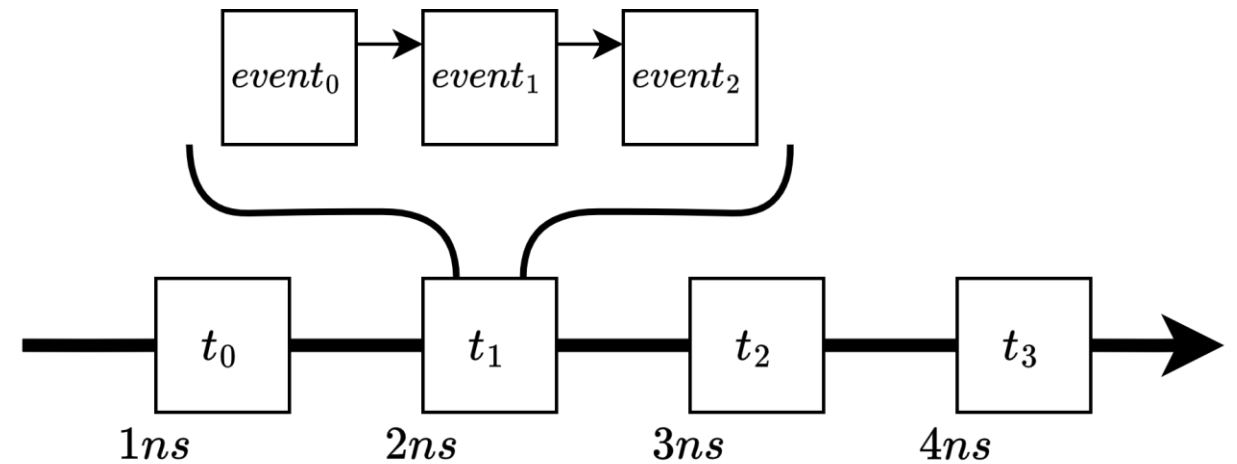
➤ Event based simulation

Example:

- Clock @ 50MHz
- Smallest time unit: 10fs
- How many steps per clock period?
- 2.000.000 steps

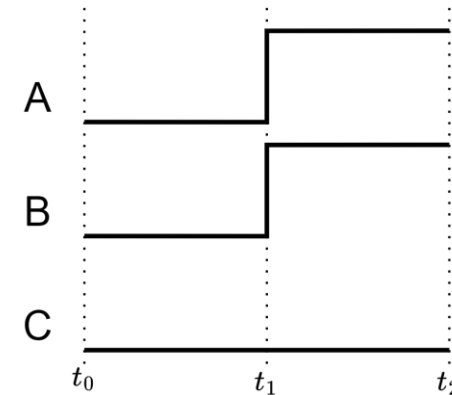
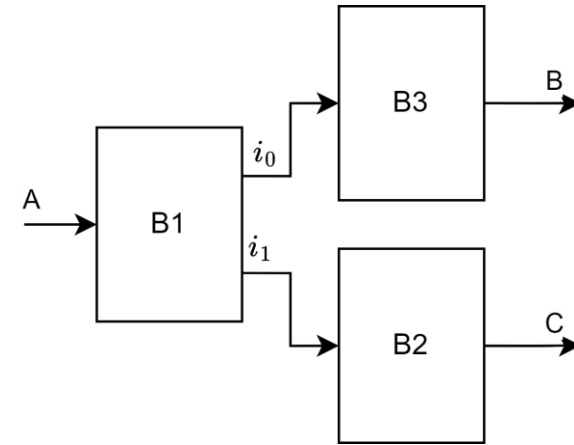
Event Based Simulation

- Signal are not evaluated at every time unit
- Instead events are scheduled, when value changes
- Events can schedule new events
- Time unit ends when all events are evaluated



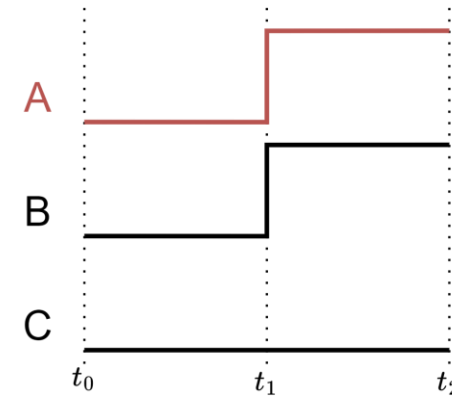
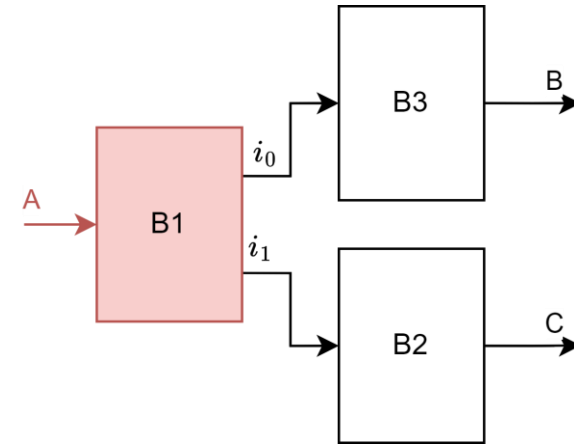
Event Based Simulation

- Signal are not evaluated at every time unit
- Instead events are scheduled, when value changes
- Events can schedule new events
- Time unit ends when all events are evaluated



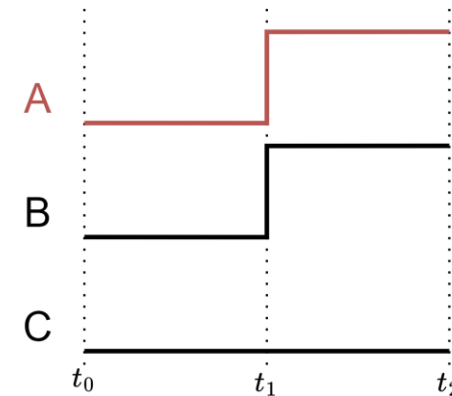
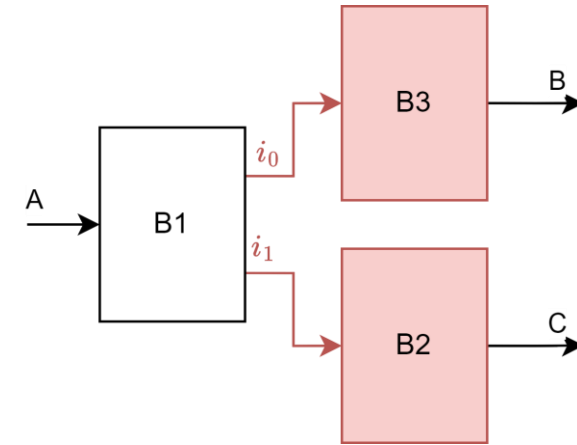
Event Based Simulation

- Signal are not evaluated at every time unit
- Instead events are scheduled, when value changes
- Events can schedule new events
- Time unit ends when all events are evaluated



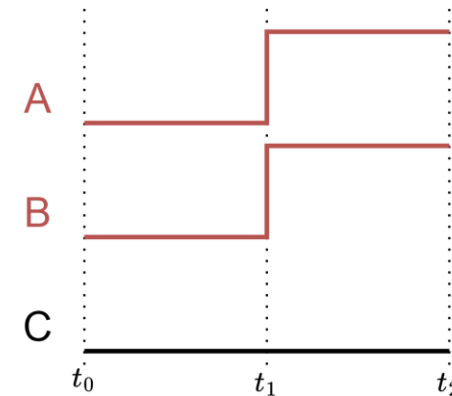
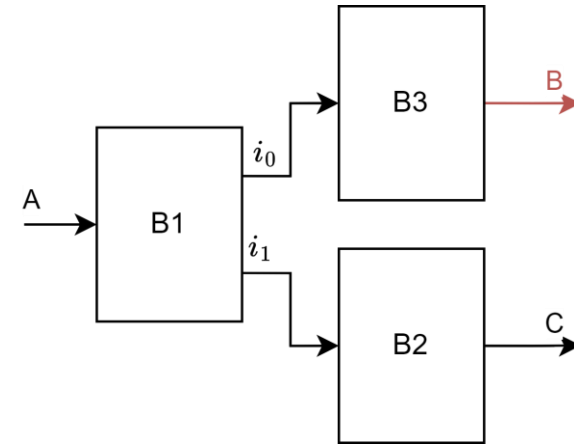
Event Based Simulation

- Signal are not evaluated at every time unit
- Instead events are scheduled, when value changes
- Events can schedule new events
- Time unit ends when all events are evaluated



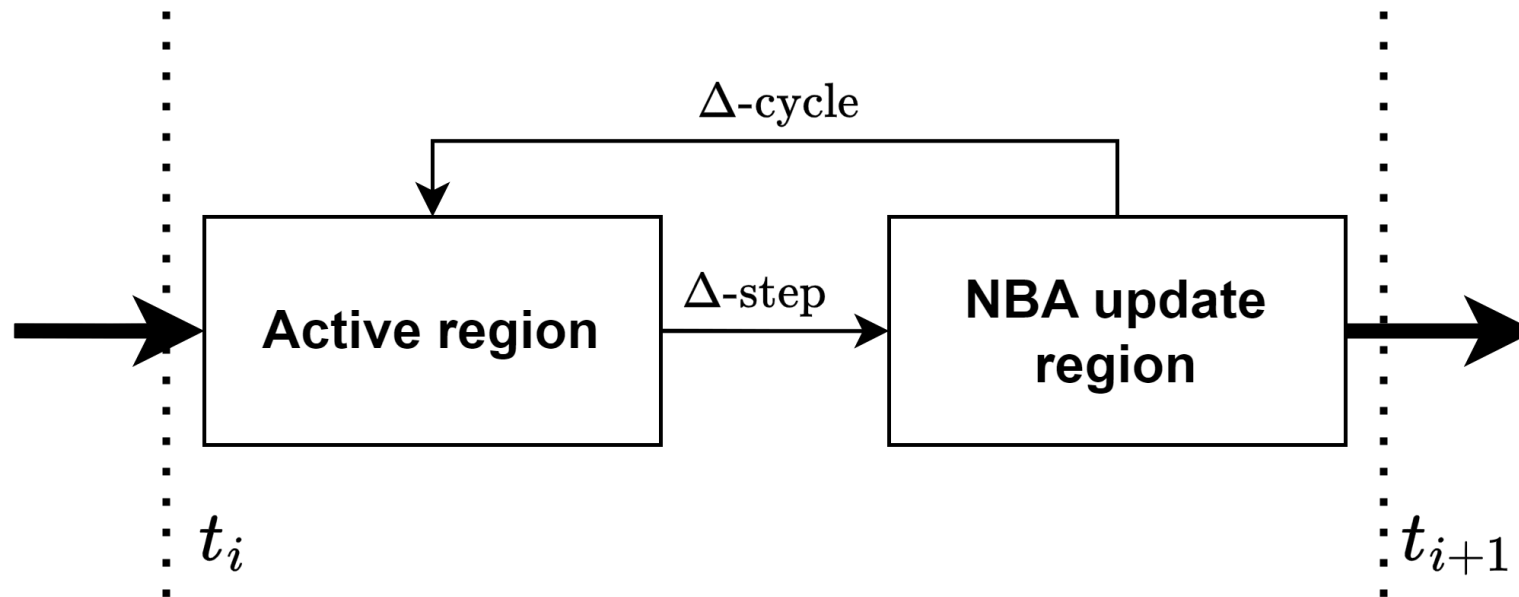
Event Based Simulation

- Signal are not evaluated at every time unit
- Instead events are scheduled, when value changes
- Events can schedule new events
- Time unit ends when all events are evaluated



Event regions/delta cycles

- Delta cycle: evaluate one region, possibly schedule more events, move on to next region, if all events evaluated and no events remaining timestep done

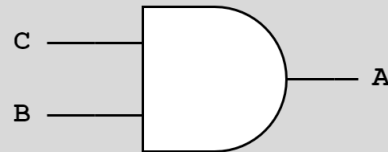


Blocking vs Non-Blocking Assignments

Blocking assignments

- Models combinational logic
- Similar to procedural C-code
- LHS and RHS evaluated at the same time (in active region)

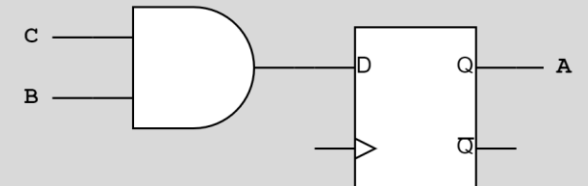
$A = B \& C$



Nonblocking assignments (NBA)

- Models sequential logic (DFF etc.)
- Functions like concurrent assignments i.e. first evaluate all RHS, then assign all LHS
- RHS evaluated in active region, LHS assigned in NBA update region

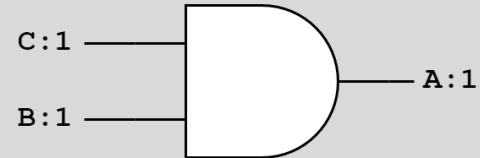
$A \leq B \& C$



Blocking vs Non-Blocking Assignments

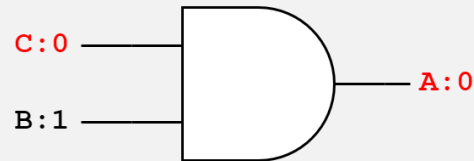
Blocking assignments

Initial:
A:1, B:1, C:1



```
<@clk>  
C = 0  
A = B & C
```

Active region

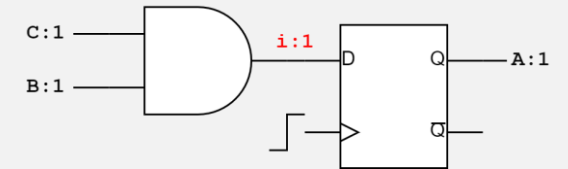


Nonblocking assignments (NBA)

Initial:
A:1, B:1, C:1

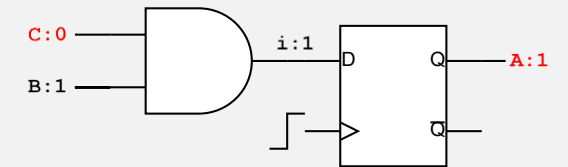
```
<@clk>  
C <= 0  
A <= B & C
```

Active region



Δ-step

NBA update region

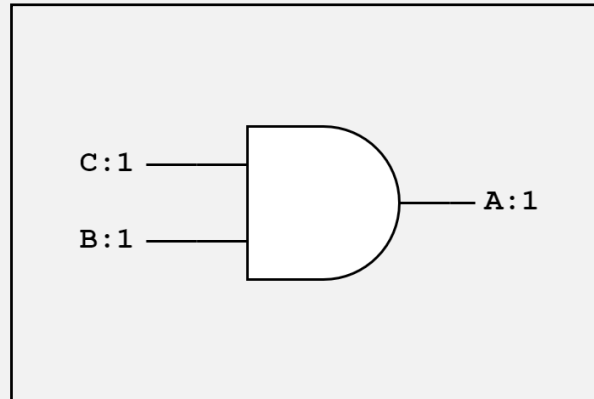


Blocking Assignments

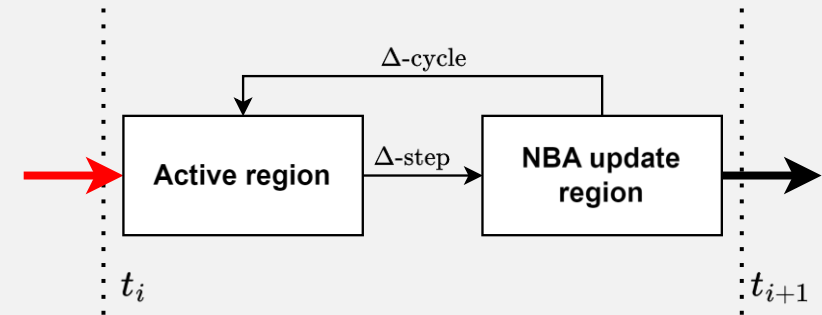
RTL

Initial state:
A:1, B:1, C:1

```
<@clk>  
C = 0  
A = B & C
```



Simulator

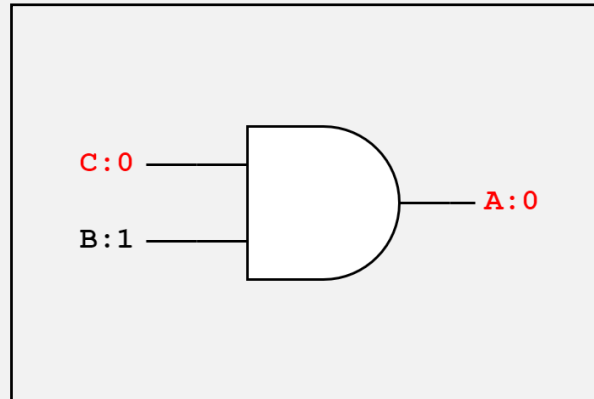


Blocking Assignments

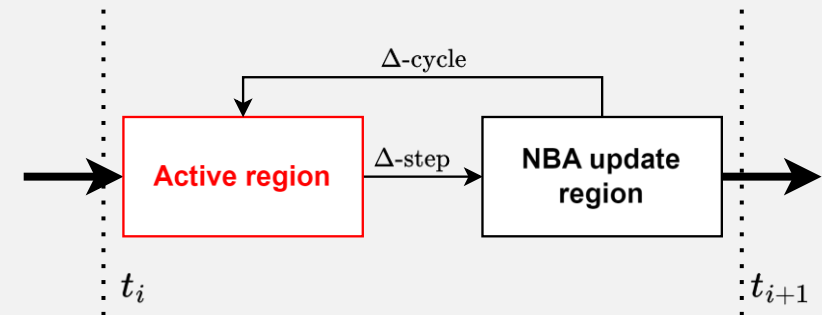
RTL

Initial state:
A:1, B:1, C:1

```
<@clk>  
C = 0  
A = B & C
```



Simulator

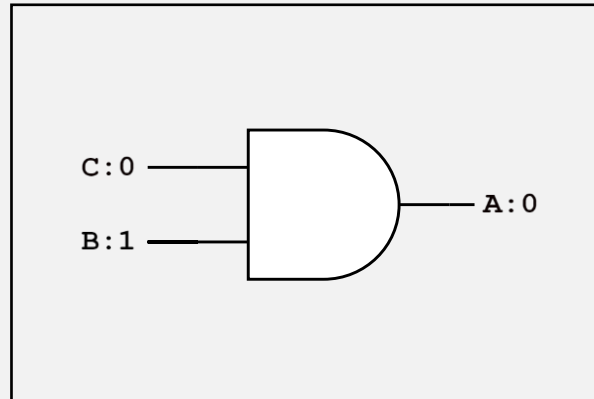


Blocking Assignments

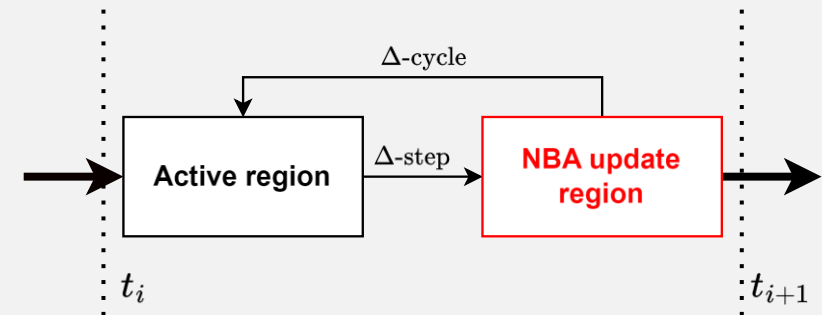
RTL

Initial state:
A:1, B:1, C:1

```
<@clk>  
C = 0  
A = B & C
```



Simulator

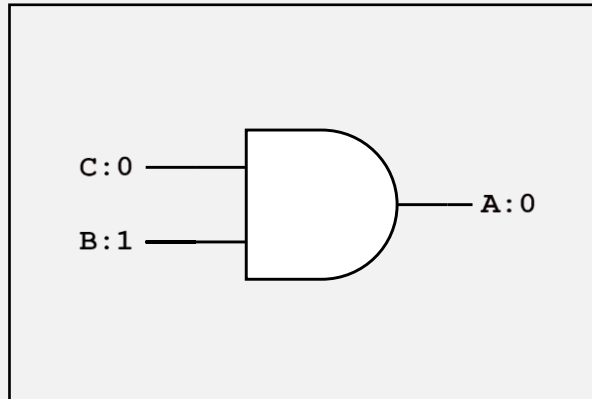


Blocking Assignments

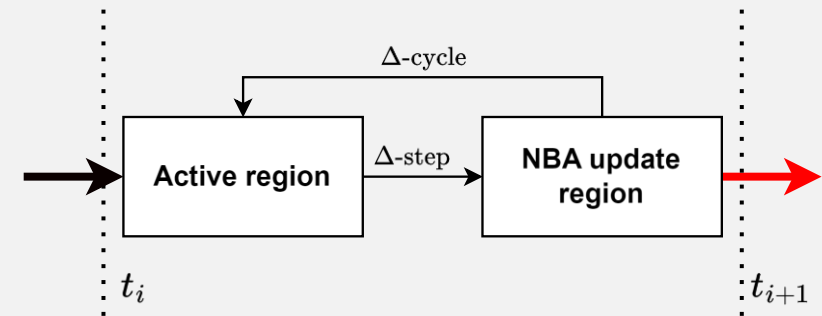
RTL

Initial state:
A:1, B:1, C:1

```
<@clk>  
C = 0  
A = B & C
```



Simulator

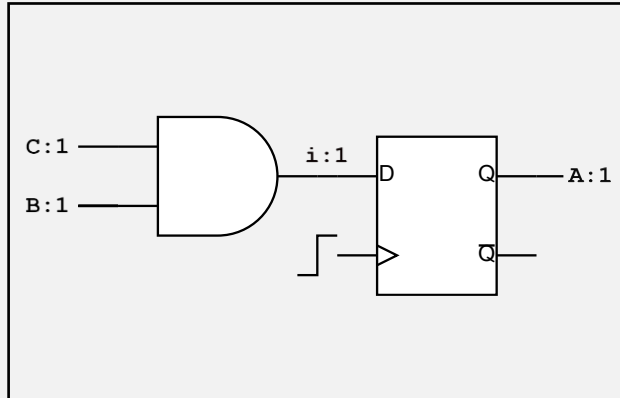


Non-Blocking Assignments

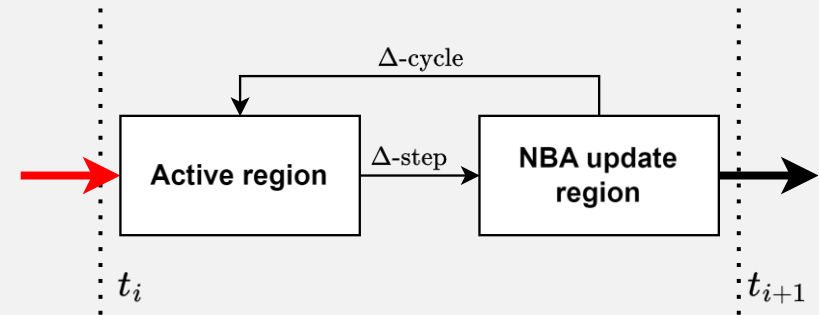
RTL

Initial state:
A:1, B:1, C:1

```
<@clk>  
C <= 0  
A <= B & C
```



Simulator

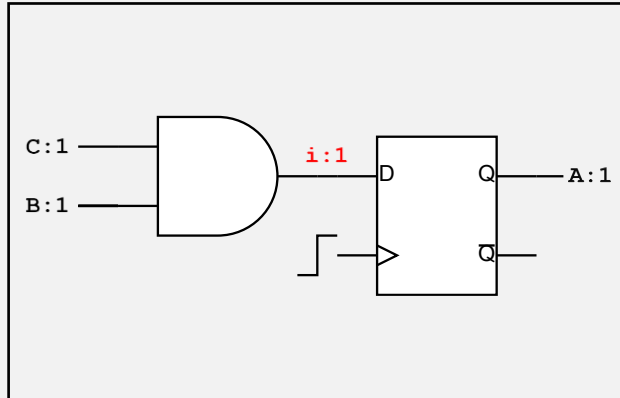


Non-Blocking Assignments

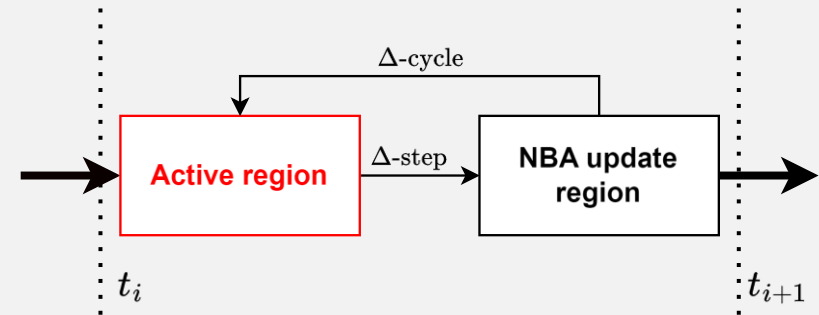
RTL

Initial state:
A:1, B:1, C:1

```
<@clk>  
C <= 0  
A <= B & C
```



Simulator

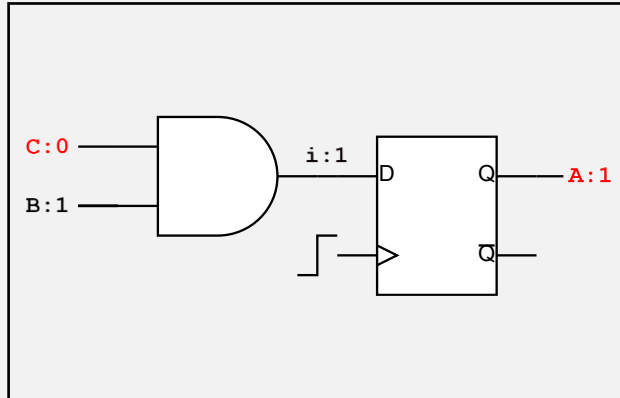


Non-Blocking Assignments

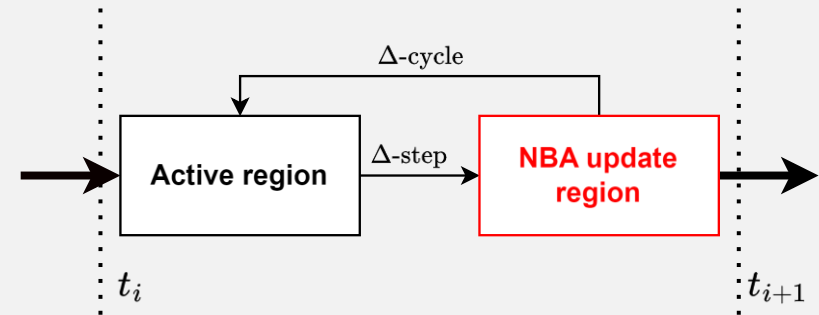
RTL

Initial state:
A:1, B:1, C:1

```
<@clk>  
C <= 0  
A <= B & C
```



Simulator

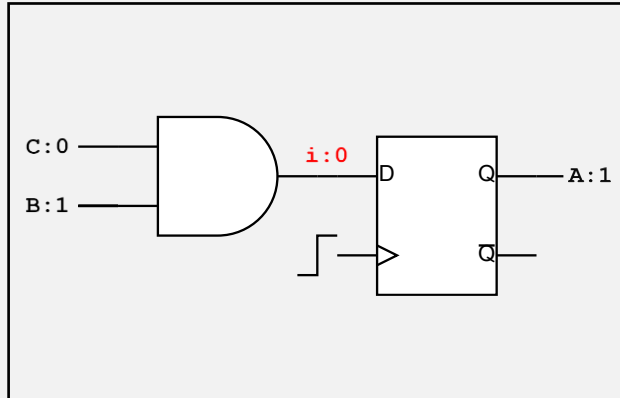


Non-Blocking Assignments

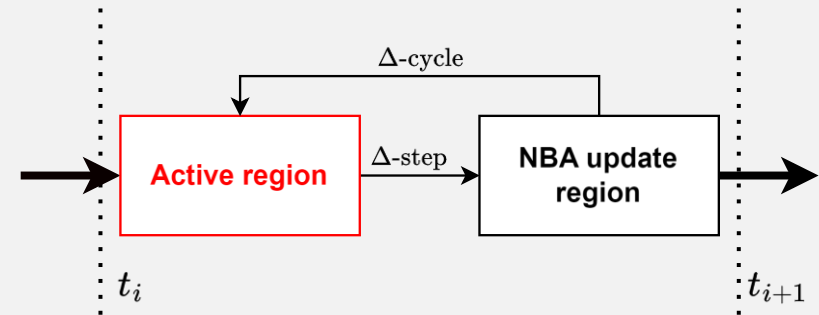
RTL

Initial state:
A:1, B:1, C:1

```
<@clk>  
C <= 0  
A <= B & C
```



Simulator

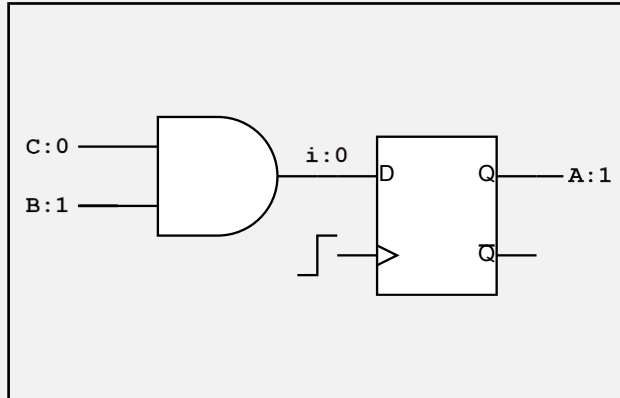


Non-Blocking Assignments

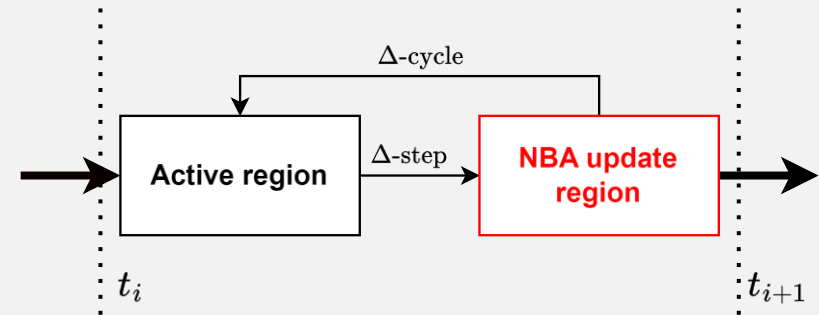
RTL

Initial state:
A:1, B:1, C:1

```
<@clk>  
C <= 0  
A <= B & C
```



Simulator

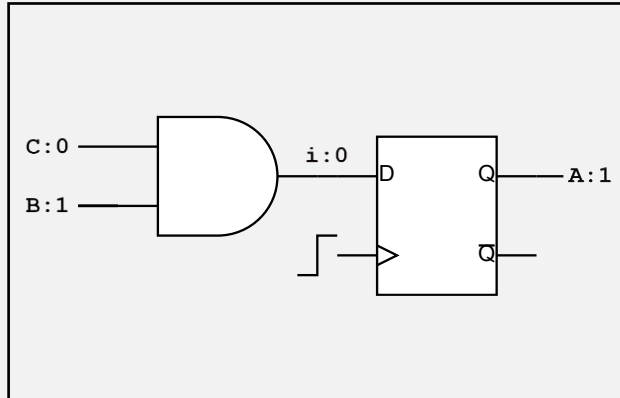


Non-Blocking Assignments

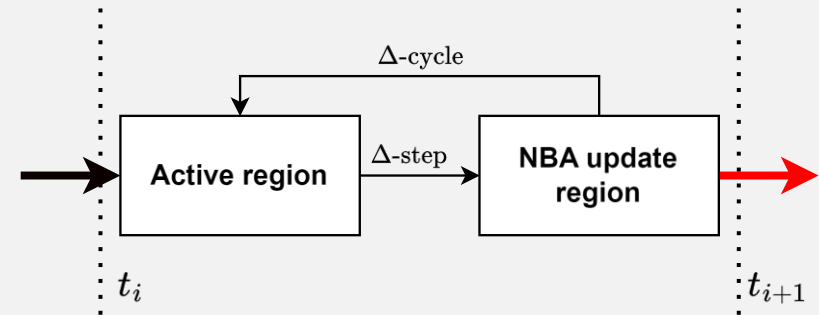
RTL

Initial state:
A:1, B:1, C:1

```
<@clk>  
C <= 0  
A <= B & C
```

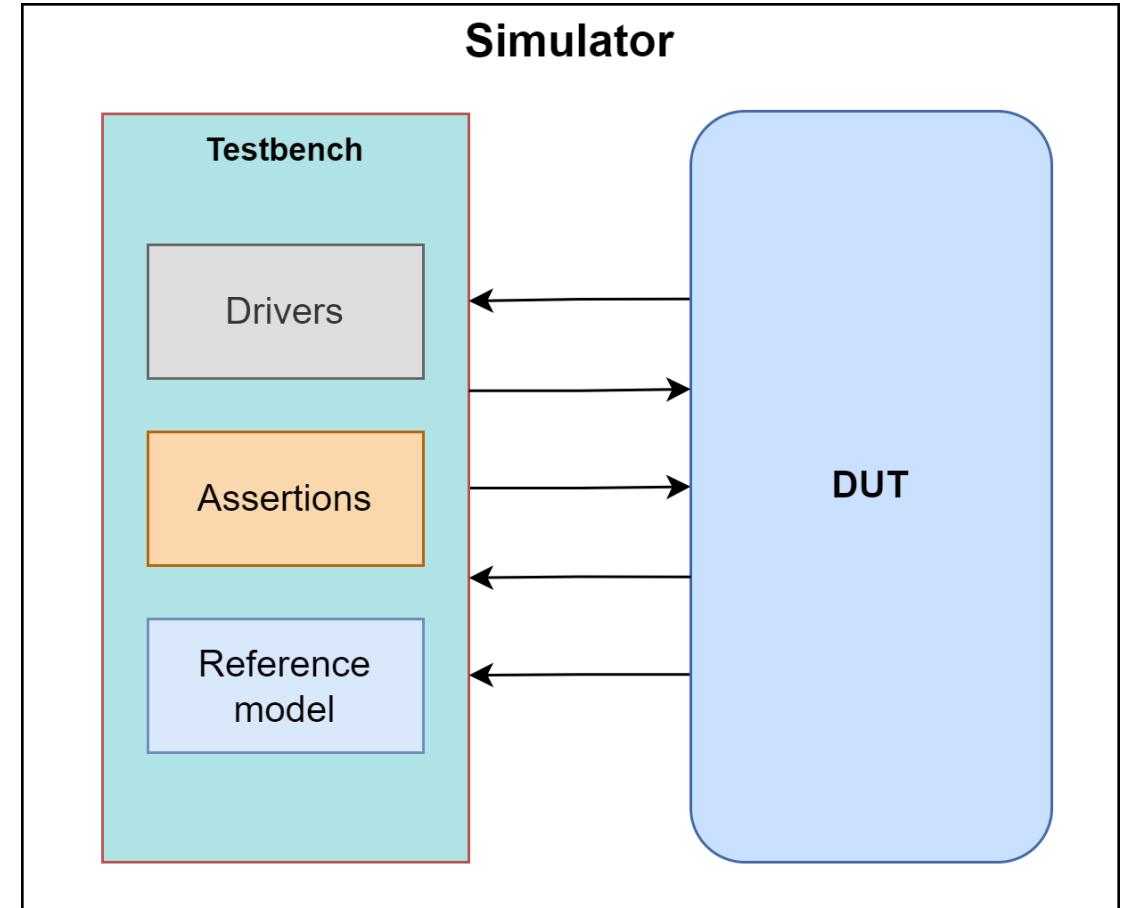


Simulator



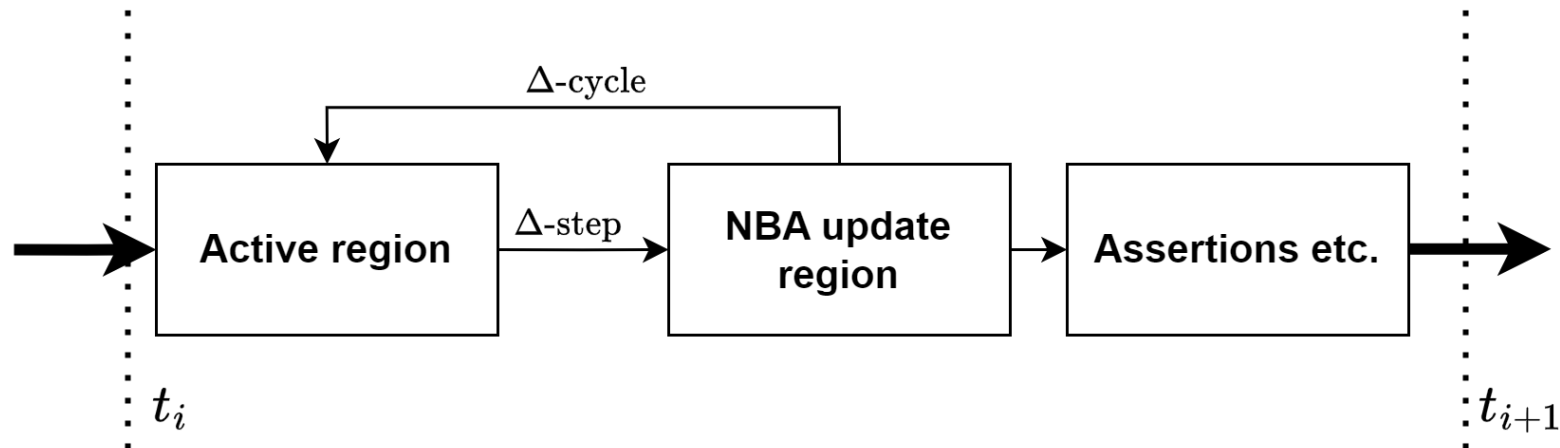
Race Conditions

- Testbench also runs in simulation cycle
- Checkers, assertions, drivers etc. might be non deterministic or rely on simulation order
- Can result in wrong/unpredictable behaviour



Checking Signals in the Test Bench

- To avoid any race conditions in the testbench
 - Always check signal values at end of timestep



Simulator Programming Interfaces

- Simulators are accessible through API to make interfacing them easier
- For (System)Verilog its VPI (Verilog Procedural Interface)
- Defined in IEEE 1364-2005
- Gives access to a number of tasks/functions to interact with the simulation and its objects